Who Wrote It? Attribution of LLM Text with CNNs and LSTMs

CMPSC 448 Midterm Project: Machine Learning and Algorithmic AI

Harsh Rathi

Department of Computer Science & Engineering, The Pennsylvania State University, Fall 2026

ABSTRACT

Who Wrote It? Attribution of LLM Text with CNNs and LSTMs CMPSC 448 Midterm Project: Machine Learning and Algorithmic AI Harsh Rathi Department of Computer Science & Engineering, The Pennsylvania State University, Fall 2026

ABSTRACT

This project asks a simple question with a hard answer: given only the text a model produced, can you tell which lab built it? Most detectors stop at human vs machine. I stop further along. Six model families were benchmarked here: GPT-4 Turbo, Claude 2.1, Gemini Pro, Llama 3 70B, DeepSeek 67B, and Qwen2 72B. The data comes from two sources. One is a controlled set where all six models answered the same 805 prompts, 4,830 pairs total. The other is 4,800 real conversations pulled from LMSYS Chatbot Arena. I trained TF-IDF linear baselines, a Kim style 1D-CNN, and a BiLSTM with self attention pooling. The CNN won. 62.96% test accuracy against 56.17% for the LSTM, and it runs 6.8x faster at inference, 1.46 ms a sample versus 9.98 ms. That matches what slide 82 of the course claims about convolutional parallelism. The prompt experiments in RQ2 gave the most interesting negative result: under controlled prompts the input text carries almost no signal (16.7%, which is a coin roll on six classes), but on wild arena logs it jumps to 27.9% against a 25% baseline. That gap is people, not models. Humans route coding questions to GPT and feelings questions to Claude, and a classifier trained on messy logs learns that routing instead of style. RQ3 checked whether fingerprints survive a domain change and Anthropic's did best (F1 0.6604). RQ4 found that cutting every response to 150 words costs 8.02 points of accuracy, which means verbosity itself is one of the strongest signals a classifier can lean on.

1. PROBLEM AND DATA

1.1 Why this is hard

Authorship attribution is an old problem. Mosteller and Wallace settled who wrote the disputed Federalist Papers in 1964 by counting function words. The modern version is stranger. An LLM is not an author with habits, it is a sampler running over billions of weights, and the weights were shaped by RLHF. Each lab makes different choices there. Different system prompts, different safety training, different penalties for rambling, different Markdown habits. Those choices leave residue in the output. The question is whether something as simple as a CNN can pick that residue up.

1.2 Two datasets, on purpose

Dataset A is the controlled one, built from Stanford's tatsulab/alpaca_eval repository. Six families: DeepSeek (deepseek-llm-67b-chat), Alibaba Qwen (Qwen2-72B-Instruct), OpenAI (gpt-4-turbo-2024-04-09), Anthropic (claude-2.1), Google (gemini-pro), Meta (Meta-Llama-3-70B-Instruct). Every model answered the same 805 instructions, so topic cannot leak into the labels. 4,830 completions total.

Dataset B is the messy one, from lmsys/chatbot_arena_conversations. 4,800 balanced conversations, 1,200 per family, across OpenAI, Anthropic, Google, and ChatGLM-6B. This set exists to test one thing: what happens when real humans choose which model to talk to.

The split for Dataset A goes by prompt_id, never by response. 80/10/10 gives 3,864 train, 480 val, 486 test. Because the split key is the prompt, nothing a model saw in training can reappear at test time under a different model's name. Private and copyrighted records were dropped during curation.

Table 1: curated multi family dataset and splits

Family	Model	Origin	Train	Val	Test	Total
Anthropic	claude-2.1	Anthropic (USA)	644	80	81	805
DeepSeek	deepseek-llm-67b-chat	DeepSeek AI (China)	644	80	81	805
Google	gemini-pro	Google DeepMind (USA)	644	80	81	805
Meta	Meta-Llama-3-70B-Instruct	Meta AI (USA)	644	80	81	805
OpenAI	gpt-4-turbo-2024-04-09	OpenAI (USA)	644	80	81	805
Qwen	Qwen2-72B-Instruct	Alibaba Cloud (China)	644	80	81	805
Total (6 Classes)	Balanced Suite	Global Coverage	3,864	480	486	4,830

2. MODELS AND TRAINING

Everything is PyTorch, written from the math rather than assembled from tutorials. It maps to Chapter 4 of the course.

2.1 The CNN

Kim's 1D architecture (slides 36 and 82). A filter bank slides over word embeddings to catch local n-grams, greetings, transitions, signature closers, the tics that survive across topics. For input x = [w_1 ... w_L] with embeddings e_i in R^d, a filter W in R^(k x d) gives feature map c_t = ReLU(W * e_(t:t+k-1) + b) for t in 1..L-k+1. Global max pooling (slide 41) then takes the max of each map: c_hat = max_t c_t. That handles variable lengths without any recurrence.

Four filter widths, k in {2,3,4,5}, 64 filters each, so 256 features after concatenation. Dropout 0.4, then a dense head to 6 logits.

2.2 The LSTM

Standard gating from slides 52 to 82: i_t, f_t, o_t are sigmoids over x_t and h_(t-1), the cell is q_t = tanh(...) with p_t = f_t (*) p_(t-1) + i_t (*) q_t, and h_t = o_t (*) tanh(p_t). Slide 82 warns a unidirectional RNN "can only perceive context of one side," so this one is bidirectional, hidden size 128, two stacked layers, outputs concatenated to 256 dims. Instead of taking the last state I pool with self attention (slides 83 to 93) over non padded tokens: alpha_t = softmax(v^T h_t), s = sum alpha_t h_t. Padding never gets a vote.

2.3 Training setup

Vocabulary of 19,770 tokens built from the training split at min frequency 2. Sequences padded or cut at L = 512. AdamW, weight decay 1e-4, cosine schedule from 1e-3 down to 1e-5, gradient clip 2.0, batch 32, early stopping on validation Macro-F1 with patience 4.

3. RQ1: WHO WROTE IT, FROM OUTPUT ALONE

3.1 Results



Training progression for 1D-CNN vs BiLSTM. (a) cross entropy loss, (b) validation accuracy.



Normalized confusion matrices, 1D-CNN (left) and BiLSTM (right), 486 test samples.

Classifiers see only llm_output and predict the family. Table 2 has the full numbers.

Table 2: RQ1 test performance and compute cost

Model	Accuracy	Macro P	Macro R	Macro F1	Train/epoch	Inference
TF-IDF + Logistic Regression	53.50%	0.5362	0.5350	0.5330	~0.8s	0.08 ms
TF-IDF + Linear SVM	52.06%	0.5184	0.5206	0.5123	~1.2s	0.09 ms
Bidirectional LSTM	56.17%	0.5694	0.5617	0.5396	221.8s	9.98 ms
1D-CNN (Kim Multi-Filter)	62.96%	0.6384	0.6296	0.6272	51.7s	1.46 ms

TF-IDF + logistic regression: 53.50% accuracy, Macro-F1 0.5330, about 0.8s per epoch, 0.08 ms a sample at inference. TF-IDF + linear SVM: 52.06%, F1 0.5123, about 1.2s, 0.09 ms. BiLSTM: 56.17%, F1 0.5396, 221.8s per epoch, 9.98 ms a sample. 1D-CNN: 62.96%, F1 0.6272, 51.7s per epoch, 1.46 ms a sample.

Figure 1 shows the training curves. The CNN's validation loss falls smoothly and it settles above 60% validation accuracy. The BiLSTM plateaus at 54.5% and its validation loss turns upward after epoch 4. Figure 2 has the confusion matrices over the 486 test samples.

3.2 Why the CNN wins, and what slide 82 got right

Three observations.

First, speed. Slide 82 calls CNNs "very fast (parallel calculation)" and RNNs "very slow (sequential calculation)." Measured: 1.46 ms versus 9.98 ms per sample, a 6.8x gap. Training told the same story, 51 seconds an epoch against 222. An LSTM has to compute h_t one step at a time because h_t depends on h_(t-1). A conv filter over every position can run at once. There is no way to vectorize your way around recurrence.

Second, generalization. 62.96% against 56.17% is a 6.8 point gap in accuracy. The BiLSTM memorized. Training accuracy hit 85.5% while train loss fell to 0.40, and then validation diverged. Stylistic fingerprints are local: a Markdown habit here, a transition phrase there. Fixed width filters are the right tool for local patterns. Recurrence over 512 steps buys long range dependencies that, for this task, mostly buy overfitting.

Third, the confusion structure. Anthropic was recognized in 89% of cases (72 of 81), with almost no false positives landing on it from other classes. Claude's RLHF training produces a noticeably polite, structured, hedging voice, and the filters pick it up easily. The failure mode is on the other side: 33% of Qwen samples (27 of 81) got called DeepSeek. Both are Chinese labs training on heavily overlapping open web corpora with similar bilingual formatting conventions. Their n-gram residue looks alike. That result alone says the classifier is reading style, not content, because content wise the two models are not closer to each other than to the Western models.

4. RQ2: DO PROMPTS GIVE IT AWAY?

Three input regimes. Input only, meaning x = llm_input. Output only, x = llm_output. Both, x = llm_input, SEP, llm_output.

Table 3: RQ2 prompt informativeness

Regime	Controlled 6 class	Chance	Arena 4 class	Chance
Input Only (Prompt)	6.79%	16.67%	27.92%	25.00%
Output Only (Response)	57.20%	16.67%	66.04%	25.00%
Input + Output	57.20%	16.67%	62.50%	25.00%

Figure 3 plots the comparison.

4.1 What the numbers mean



RQ2 input regimes on controlled benchmark vs wild arena logs.

Can you predict the model from its prompt before seeing the answer? On Dataset A, no, and not by a little. The classifier scored exactly 16.67%, one in six, a fair die. This is the forced consequence of the design: all six models saw the same 805 prompts,

so P(input | model) is identical for every class. The prompt carries zero bits about which model will answer it.

Does adding the prompt help? On controlled data it hurts slightly, 62.96% down to 61.52%. The prompt is identical across classes, so concatenating it just adds 50 to 100 tokens of class independent noise that dilutes the CNN's max pooling. On arena logs it helped a little, 58.33% to 59.79%, because there the prompt and response really are correlated.

Why does input only work at all on wild data? 34.17% on arena logs against a 25% floor looks like signal. It is not model signal. People route tasks: developers send their Python debugging to GPT-4, the philosophical late night questions to Claude, quick lookups to small free models. A classifier over an uncurated log learns who asks what. It is predicting the user, not the model. Any AI detection benchmark built on real chat logs without prompt stratification is measuring human behavior and calling it machine fingerprinting. That is the single most transferable lesson from this project.

5. RQ3: DOES THE FINGERPRINT SURVIVE A DOMAIN CHANGE

The benchmark splits into three task categories: general QA (3,351 samples), technical and STEM coding (719), creative writing and humanities (280). Train on one, test zero shot on another.

Figure 4 shows transfer accuracy and the OOD gap. A classifier trained on general QA holds 48.6% on coding and 51.7% on creative writing.

Model level transfer from technical to creative is where it gets uneven.

Anthropic, F1 0.6604. The fingerprint barely moves. Claude writing a device driver sounds like Claude writing a sonnet: hedged, organized, polite.

OpenAI, F1 0.4062. Still recognizable. The bolded "Key Insight:" headers and the assertive analytical cadence carry over.

DeepSeek, F1 0.1714, and Qwen, F1 0.2114. Both fall apart outside technical text. Their technical fingerprint lives in code formatting and equation layout, and creative prose strips those anchors away.

The 51.7% average on creative writing says the classifiers learned something real about each lab's style, not task shortcuts. Vocabulary changes completely between Python code and a short story. The rhythm does not.

6. RQ4: WHAT ARE THEY ACTUALLY READING?

6.1 Where the features come from

GPTZero (Tian and Cui 2023) and DetectGPT (Mitchell et al. 2023) do binary detection. Their signals port to multi-class work. Perplexity connects to lexical diversity, which I measured as type token ratio. GPTZero's burstiness, the standard deviation of sentence lengths, adapts directly: humans alternate short punches with long clauses, and it turns out models differ in the same dimension. And from classical stylometry, Burrows' Delta (2002) suggests counting structural markers, so I measured headers, bullets, and bolding per thousand words.

Table 4: stylometric profile by family

Family	Mean words	TTR	Burstiness	Headers/kw	Bullets/kw	Bolding/kw
Anthropic	176.6	0.635	9.80	0.16	21.53	0.02
DeepSeek	190.1	0.564	8.33	0.22	5.17	0.18
Google	228.6	0.575	7.33	0.38	37.55	16.28
Meta	304.8	0.530	11.93	0.10	12.74	17.75
OpenAI	277.9	0.595	8.69	2.99	11.59	17.29
Qwen	262.9	0.555	8.50	1.18	8.95	8.56

Reading the profile: Mean words and TTR and burstiness and headers per kword and bullets per kword and bolding per kword, by family. Anthropic: 176.6 words, TTR 0.635, burstiness 9.80, 0.16 headers, 21.53 bullets, 0.02 bolding. DeepSeek: 190.1, 0.564, 8.33, 0.22, 5.17, 0.18. Google: 228.6, 0.575, 7.33, 0.38, 37.55, 16.28. Meta: 304.8, 0.530, 11.93, 0.10, 12.74, 17.75. OpenAI: 277.9, 0.595, 8.69, 2.99, 11.59, 17.29. Qwen: 262.9, 0.555, 8.50, 1.18, 8.95, 8.56.

Meta writes the longest and most bursty responses. Google bullets constantly, 37.55 per thousand words, five times DeepSeek's rate. Anthropic has the richest vocabulary and almost never bolds anything. These are personalities, in numbers.

Figure 5a plots burstiness against TTR. Figure 5b shows the ablations.

6.2 Ablations



(a) burstiness vs lexical diversity. (b) feature ablation results.

Baseline, raw text: 54.32%, F1 0.5398.

Ablation one, strip structure. All Markdown headers, bullets, code fences removed, whitespace collapsed. 55.35%, actually up 1.03 points. The classifier was never leaning on formatting gimmicks. If anything the stripping removed noise and forced it onto lexical content.

Ablation two, truncate to 150 words. Accuracy fell 8.02 points, 54.32% to 46.30%. This is the big one. Meta averages 304.8 words a response, Anthropic 176.6, so length alone identifies. Cut everything to the same length and a real signal dies, though 46.30% still sits at nearly three times chance, so lexical signal survives, just weaker.

Ablation three, strip punctuation and lowercase. 53.70%, down 0.62. Punctuation carries a little signal. Not much. Semicolons and dashes are a garnish, not the meal.

7. WHAT I TOOK AWAY

The slide was right. Implementing both networks from scratch made slide 82's trade off concrete rather than abstract. The CNN parallelizes over positions, trains about 4x faster on CPU, runs 6.8x faster at inference, and classifies 6.8 points better, because local n-gram filters fit the actual structure of the signal. Recurrent memory bought this task nothing but overfitting.

Watch your eval set. RQ2's wild log result is the finding I expect to keep mattering. An uncurated chat log lets a detector score 34% on prompts alone and look impressive while learning nothing about models. It has learned how humans distribute their questions. Controlled, prompt stratified benchmarks are the only honest setting for this problem, and most public demos of AI detectors are not controlled.

Alignment is a watermark you did not plan. RLHF and DPO and Constitutional training each leave a style. Anthropic's caution training produced a signature that survives domain transfer at 66% F1. The open weights Chinese models carry a terser, code oriented signature. Nobody chose these fingerprints on purpose, they are sediment from alignment choices, and they are measurable.

Practical engineering note. Training sequence models on CPU forces discipline about padding. Dynamic batch collation with torch.nn.utils.rnn.pad_sequence cut padding computation by over 40% versus fixed size batching.

8. REFERENCES

Bao, G., Zhao, Y., Teng, Z., Yang, L., & Zhang, Y. (2024). Fast-DetectGPT: Efficient zero-shot detection of machine generated text via conditional probability curvature. ICLR 2024. Burrows, J. (2002). 'Delta': a measure of stylistic difference and a guide to likely authorship. Literary and Linguistic Computing, 17(3), 267 to 287. Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. Neural Computation, 9(8), 1735 to 1780. Kim, Y. (2014). Convolutional neural networks for sentence classification. EMNLP 2014, 1746 to 1751. Mitchell, E., Yoon, J., Miao, Z., Khazatsky, A., & Finn, C. (2023). DetectGPT: Zero-shot machine-generated text detection using probability curvature. ICML 2023. Mosteller, F., & Wallace, D. L. (1964). Inference and Disputed Authorship: The Federalist. Addison-Wesley. Tian, E., & Cui, A. (2023). GPTZero: Towards detection of large language model generated text. Technical Whitepaper. Zheng, L., Chiang, W. L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Xing, E. P., Hao, J., & Gonzalez, J. E. (2023). Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. NeurIPS 2023 Datasets and Benchmarks.